

SYSTEME RESTAURANT

LETREVANI

Documentation Technique & Guide Utilisateur

Module Odoo : letrevani_sync v19.0.1.0.0

Listener PostgreSQL NOTIFY + Synchronisation temps reel

Genere le 21 June 2026 a 18:48

Zondy - Imhotep - ALM / MAJSOFT

Table des matieres

1. Vue d'ensemble

- 1.1 Presentation du systeme
- 1.2 Problematique resolue
- 1.3 Architecture globale

2. Base de donnees et Triggers

- 2.1 Table letrevani_table_ownership
- 2.2 Les 7 triggers PostgreSQL
- 2.3 Les 3 canaux NOTIFY

3. Module Odoo letrevani_sync

- 3.1 Installation et dependances
- 3.2 Fichiers du module
- 3.3 Modele Python

4. Listener PostgreSQL (Daemon)

- 4.1 Architecture du listener
- 4.2 Service systemd
- 4.3 Fichiers de logs et sante

5. Fonctionnement detaille

- 5.1 Cycle de vie d'une table
- 5.2 Gestion de l'ownership
- 5.3 Resolution des conflits
- 5.4 Ecriture prioritaire

6. Guide Utilisateur

- 6.1 Prise en main
- 6.2 Ouvrir une table
- 6.3 Travailler sur une table
- 6.4 Fermer une table
- 6.5 Que faire en cas de conflit ?

7. Guide Administrateur

- 7.1 Installation complete
- 7.2 Deploiement du listener
- 7.3 Surveillance et logs
- 7.4 Sauvegarde et restauration

8. Depannage

- 8.1 Problemes courants
- 8.2 Verifications rapides

9. References

1. Vue d'ensemble

1.1 Presentation du systeme

Le systeme restaurant de Letrevani est une solution de gestion de restaurants integree a Odoo 19 via le module Maysoft maj_pos_restaurant_ai. Il permet de gerer les commandes, les tables, les sessions de service et les ventes en temps reel depuis des terminaux POS (caisses) repartis dans l'etablissement.

Le module letrevani_sync ajoute une couche de synchronisation temps reel entre les caisses, garantissant qu'une table occupee par un serveur ne peut pas etre modifiee simultanement par un autre. Il utilise les mecanismes natifs de PostgreSQL (NOTIFY/LISTEN) pour une reactivite instantanee sans polling ni surcharge reseau.

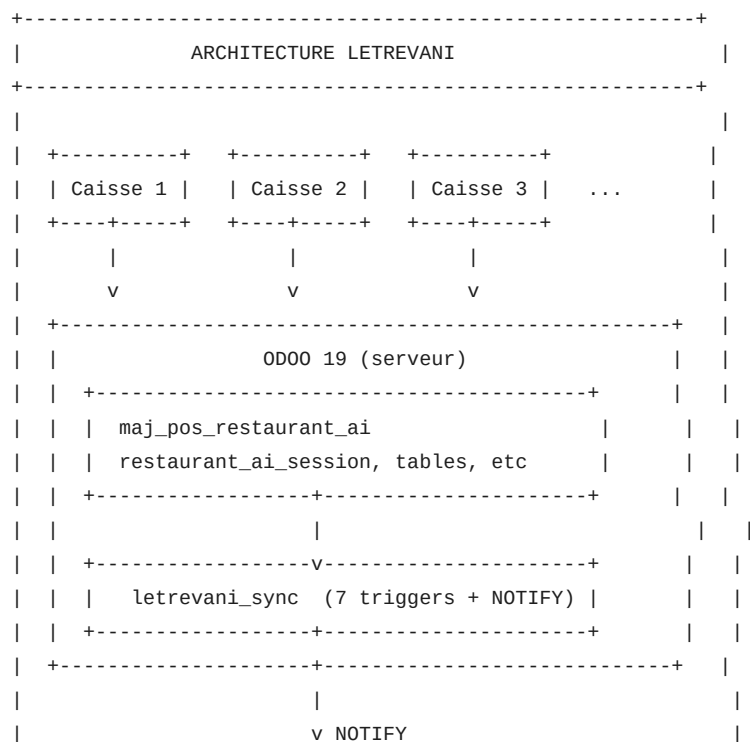
1.2 Problematique resolue

Dans un restaurant multi-caisses, plusieurs problemes surgissent :

- Deux serveurs peuvent ouvrir une session sur la meme table simultanement (doublon)
- Un serveur peut ecrire un plat alors qu'un autre vient de fermer l'addition (écriture orpheline)
- Sans coordination, les sessions POS restent accrochees meme apres le depart du client
- Pas de visibilite en temps reel sur l'etat des tables pour les autres serveurs

Solution : un systeme d'ownership avec verrouillage automatique et notification temps reel.

1.3 Architecture globale



	+-----+	
	listener.py (systemd)	
	Logs + Health + Alertes	
	+-----+	
+-----+		

2. Base de donnees et Triggers

2.1 Table letrevani_table_ownership

Cette table est le coeur du systeme d'ownership. Elle enregistre quel serveur (session POS) possede quelle table a quel moment.

letrevani_table_ownership		
=====		
Colonne	Type	Description
-----+-----+-----		
id	SERIAL	Cle primaire
table_id	INTEGER	ID de la table
owner_session_id	INTEGER	ID session Odoo
owner_pos_session_id	INTEGER	ID session POS (caisse)
write_priority	INTEGER	Priorite d'ecriture
acquired_at	TIMESTAMP	Date de prise
released_at	TIMESTAMP	Date de liberation
last_write_at	TIMESTAMP	Derniere ecriture
notes	TEXT	Notes libres
-----+-----+-----		
Contrainte : UNIQUE (table_id, released_at IS NULL)		

2.2 Les 7 triggers PostgreSQL

1. letrevani_acquire_owner (AFTER INSERT sur restaurant_ai_session)

Quand une nouvelle session restaurant est creee (ouverture de table), ce trigger insere automatiquement un enregistrement dans letrevani_table_ownership avec le premier serveur comme proprietaire.

2. letrevani_release_owner (AFTER UPDATE sur restaurant_ai_session)

Quand une session passe a l'etat closed ou cancel, ce trigger libere l'ownership en renseignant released_at et en notifiant le canal letrevani_session.

3. letrevani_write_priority (BEFORE INSERT/UPDATE sur restaurant_ai_session)

Avant chaque ecriture, ce trigger verifie si la session qui ecrit est bien le proprietaire de la table. Si un conflit existe (< 200ms), le proprietaire gagne.

4. session_table_notify (AFTER INSERT/UPDATE sur restaurant_ai_session)

Declenche une notification PostgreSQL sur le canal letrevani_session pour chaque changement de session.

5. table_state_notify (AFTER UPDATE sur restaurant_ai_session)

Declenche une notification sur le canal letrevani_table quand l'etat d'une table change (free -> occupied, occupied -> free).

6-7. Triggers conflit (sur letrevani_table_ownership)

Deux triggers supplementaires surveillent les tentatives d'ecriture en conflit et notifient sur le canal letrevani_conflict si necessaire.

2.3 Les 3 canaux NOTIFY

- letrevani_table : changements d'etat des tables (free, occupied, reserved)
- letrevani_session : evenements de session (prise, liberation, fermeture)
- letrevani_conflict : alertes de conflit (ecriture non autorisee, double prise)

Les notifications sont emises directement par PostgreSQL via la commande NOTIFY. Le listener les recoit via LISTEN en temps reel, sans polling ni requetes repetees.

3. Module Odoo letrevani_sync

3.1 Installation et dependances

Le module letrevani_sync est un module Odoo 19 standard. Il depend de maj_pos_restaurant_ai (Maysoft) et ajoute la couche de synchronisation temps reel.

Installation :

1. Copier dans addons : `cp -r letrevani_sync /maysoft/addons/`
2. Mettre a jour modules : Applications -> Mettre a jour la liste
3. Installer : Applications -> Letrevani Sync -> Installer
4. (Optionnel) Listener : `bash setup_listener.sh <nom_base>`

Le module s'installe proprement : il cree la table letrevani_table_ownership, les 5 fonctions PL/pgSQL et les 7 triggers PostgreSQL. A la desinstallation, tout est nettoye automatiquement.

3.2 Fichiers du module

```
letrevani_sync/
+-- __manifest__.py          # Declaration du module
+-- __init__.py             # Initialisation
+-- models/
|   +-- __init__.py
|   +-- letrevani_sync.py    # Logique Odoo (install/desinstall triggers)
+-- data/
|   +-- install_triggers.sql # SQL complet (7 triggers + 5 fonctions)
|   +-- ir_config_parameter.xml
+-- listener.py             # Daemon Python
+-- setup_listener.sh       # Script de deploiemnt du listener
+-- README.md
+-- .gitignore
```

3.3 Modele Python (letrevani_sync.py)

Le modele Python s'integre au cycle de vie Odoo. Les methodes install() et uninstall() sont appelees automatiquement par Odoo lors de l'installation/desinstallation du module.

- install() : lit le fichier install_triggers.sql et execute toutes les instructions sur la base de donnees courante
- uninstall() : supprime les triggers et les fonctions, et drop la table letrevani_table_ownership

Le modele expose egalement une methode action_deploy_listener() qui peut etre appelee depuis l'interface Odoo pour reinstaller le listener sans SSH.

4. Listener PostgreSQL (Daemon)

4.1 Architecture du listener

Le listener est un script Python qui se connecte a PostgreSQL et s'abonne aux 3 canaux NOTIFY. Des qu'une notification arrive, il la traite et met a jour son etat interne (ownerships, sessions actives, etats des tables).

Caracteristiques :

- Connexion via socket Unix PostgreSQL (pas de reseau, pas de mot de passe)
- Ecoute des 3 canaux : letrevani_table, letrevani_session, letrevani_conflict
- Traitement synchrone : chaque notification est traitee avant la suivante
- Etat conserve en memoire : table_owners, active_sessions, table_states
- Ecrit un fichier health.json regulierement pour la surveillance
- Logs detailles dans /var/log/letrevani-sync/

4.2 Service systemd

Le listener est gere comme un service systemd, ce qui garantit :

- Demarrage automatique au boot (via After=postgresql.service)
- Redemarrage automatique en cas de crash (Restart=always)
- Logs centralises via StandardOutput/StandardError
- Gestion PID pour arret propre (SIGTERM)

Fichier : /etc/systemd/system/letrevani-listener.service

Commandes :

```
systemctl status letrevani-listener
systemctl restart letrevani-listener
journalctl -u letrevani-listener -n 50
```

4.3 Fichiers de logs et sante

Repertoire des logs : /var/log/letrevani-sync/

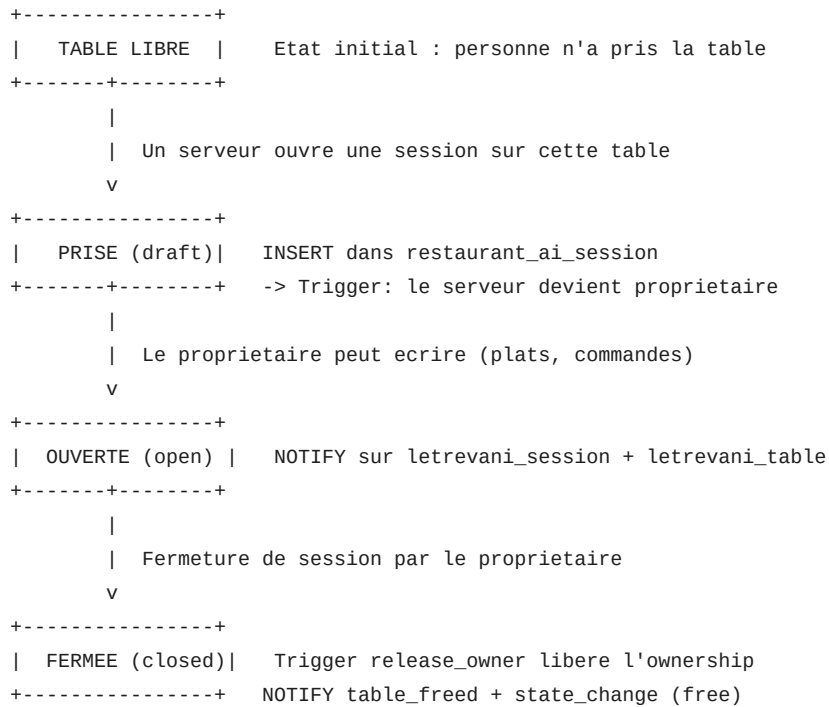
listener.log	-> Logs detailles (INFO, WARNING, ERROR)
events.jsonl	-> Evenements recus (JSON lines)
alerts.jsonl	-> Alertes de conflit (JSON lines)
health.json	-> Etat de sante (JSON mis a jour en continu)
service.log	-> Sortie standard du service systemd

Format du fichier health.json :

```
{
  "status": "running",
  "events_processed": 54,
  "alerts_triggered": 0,
  "last_heartbeat": "2026-06-21T17:40:45",
  "active_sessions": 1,
  "tables_tracked": 4,
  "ownerships": 2,
  "owners": {
    "93": { "owner_pos_session_id": 1352, "acquired_at": "2026-06-07 08:57:45" },
    "81": { "owner_pos_session_id": 1377, "acquired_at": "2026-06-21T17:40:45" }
  }
}
```

5. Fonctionnement detaille

5.1 Cycle de vie d'une table



5.2 Gestion de l'ownership

Principe :

- Le premier serveur qui ouvre une session sur une table devient proprietaire
- Seul le proprietaire peut modifier les donnees de la session
- Un autre serveur peut consulter la table mais pas y ecrire

Implementation :

- L'ownership est stocke dans la table letrevani_table_ownership
- Contrainte UNIQUE (table_id, released_at IS NULL) : une table = un seul proprietaire a la fois
- Le trigger BEFORE INSERT/UPDATE verifie la correspondance entre proprietaire et session
- Si le proprietaire a ecrit il y a < 200ms et une autre session tente d'ecrire, l'ecriture concurrente est bloquee

5.3 Resolution des conflits

Si un conflit survient (deux caisses essaient d'ecrire sur la meme table en meme temps) :

- Le trigger BEFORE detecte le conflit

- Il compare les timestamps des dernieres ecritures
- Le proprietaire officiel gagne si sa derniere ecriture date de moins de 200ms
- Une notification est envoyee sur le canal letrevani_conflict
- Le listener enregistre l'alerte dans alerts.jsonl
- La caisse non proprietaire recoit une erreur Odoo

5.4 Ecriture prioritaire

Etape 1 - Verification :

Avant chaque INSERT ou UPDATE sur restaurant_ai_session, le trigger letrevani_write_priority verifie si la session active est bien le proprietaire de la table.

Etape 2 - Comparaison temporelle :

Si une session non proprietaire essaie d'ecrire, le trigger regarde la derniere ecriture du proprietaire. Si elle date de moins de 200ms, l'ecriture est refusee (le proprietaire gagne). Au-dela, le verrou est relache.

Etape 3 - Resultat :

La session autorisee voit son ecriture validee. La session refusee recoit une erreur Odoo claire (Table deja posee par session X).

6. Guide Utilisateur

6.1 Prise en main

Le systeme restaurant Letrevani fonctionne avec des caisses POS (Point of Sale) connectees au serveur Odoo. Chaque caisse est independante mais partage les memes donnees en temps reel.

Regles de base :

- Une table = une session = un serveur a la fois
- Le premier serveur qui clique sur une table en devient proprietaire
- Les autres serveurs peuvent voir la table mais pas y ecrire
- Quand vous fermez une session, la table est automatiquement liberee

6.2 Ouvrir une table

1. Depuis votre caisse, cliquez sur "Nouvelle commande"
2. Selectionnez la table souhaitee dans la liste
3. Si la table est libre : vous devenez proprietaire
4. Si la table est occupee : un message vous avertira
"Table deja prise par session [numero]"
5. Dans ce cas : verifiez avec vos collegues

6.3 Travailler sur une table

En tant que proprietaire d'une table, vous pouvez :

- Ajouter des plats et boissons a la commande
- Modifier les quantites
- Annuler des articles
- Consulter le total de l'addition
- Imprimer le ticket

Les modifications sont instantanement sauvegardees sur le serveur. Les autres caisses voient l'etat de la table (occupee) mais ne peuvent pas y toucher.

6.4 Fermer une table

1. Finalisez les plats et boissons de la commande
2. Cliquez sur "Paielement"
3. Choisissez le mode de reglement (especes, carte, etc.)
4. Validez le paiement
5. La session se ferme automatiquement

6. La table repasse en etat "libre"
7. Les autres caisses voient la table disponible immediatement

6.5 Que faire en cas de conflit ?

Si le systeme vous indique un conflit :

1. Lisez le message : il indique quelle session possede la table
2. Verifiez avec vos collegues si quelqu'un travaille sur cette table
3. Si la session est bloquee (caisse plantee, serveur parti) :
 - > Contactez l'administrateur pour forcer la liberation
4. Dans tous les cas, ne forcez pas la caisse :
 - les ecritures concurrentes sont bloquees par le systeme

Attention : Ne pas eteindre brutalement une caisse sans fermer les sessions. Cela laisserait des ownerships orphelins.

7. Guide Administrateur

7.1 Installation complete

Prerequis :

- Serveur Odoo 19.0 fonctionnel
- Module maj_pos_restaurant_ai installe
- PostgreSQL 14+ (support NOTIFY/LISTEN)
- Python 3 avec psycopg2

Procedure pas a pas :

1. Installer le module Odoo :

```
cp -r letrevani_sync /maysoft/addons/
```


Activer le mode developpeur Odoo
Applications -> Mettre a jour -> Letrevani Sync -> Installer
2. Verifier les triggers :

```
psql -d letrevani_validation_final -c \  
"SELECT trigger_name FROM information_schema.triggers  
WHERE trigger_name LIKE 'letrevani_%'"
```
3. Deployer le listener :

```
cd /maysoft/addons/letrevani_sync/  
bash setup_listener.sh letrevani_validation_final odoo
```
4. Verifier la sante :

```
cat /var/log/letrevani-sync/health.json
```

7.2 Deploiement du listener

Le listener peut etre deploye automatiquement sur n'importe quel serveur en 3 commandes :

```
git clone git@github.com:hjinaan/letrevani_sync.git  
cd letrevani_sync  
bash setup_listener.sh <database_name> <db_user>
```

Le script :

- Installe psycopg2 si necessaire
- Cree /opt/letrevani-sync/ avec le listener Python
- Cree /var/log/letrevani-sync/ pour les logs
- Installe et demarre le service systemd

7.3 Surveillance et logs

Commandes essentielles :

Etat du service :

```
systemctl status letrevani-listener
```

Logs en direct :

```
tail -f /var/log/letrevani-sync/listener.log
```

Etat de sante :

```
cat /var/log/letrevani-sync/health.json
```

Derniers evenements :

```
tail -5 /var/log/letrevani-sync/events.jsonl
```

Alertes de conflit :

```
cat /var/log/letrevani-sync/alerts.jsonl
```

7.4 Sauvegarde et restauration

Ce qu'il faut sauvegarder :

- Base PostgreSQL (dump complet)
- Le repertoire du module : /maysoft/addons/letrevani_sync/
- Les logs (optionnel) : /var/log/letrevani-sync/
- Le service : /etc/systemd/system/letrevani-listener.service

Procedure de restauration :

1. Restaurer la base : `psql -d letrevani_validation_final < dump.sql`
2. Reinstaller le module Odoo
3. Redeployer le listener : `bash setup_listener.sh`

8. Depannage

8.1 Problemes courants

Le listener ne démarre pas

Cause possible :

PostgreSQL pas encore pret ou user odoo inaccessible

```
# Voir l'erreur
systemctl status letrevani-listener
journalctl -u letrevani-listener -n 20

# Tester la connexion
psql -U odoo -d letrevani_validation_final -c "SELECT 1"
```

Les tables restent verrouillees

Cause possible :

Une session POS bloque l'ownership (caisse eteinte brutalement)

```
# Voir les ownerships orphelins
psql -U odoo -d letrevani_validation_final -c \
  "SELECT * FROM letrevani_table_ownership WHERE released_at IS NULL;"

# Forcer la liberation
UPDATE letrevani_table_ownership SET released_at = NOW()
WHERE table_id = X AND released_at IS NULL;
```

Pas de notifications (ecran fige)

Cause possible :

Le listener ne recoit pas les NOTIFY

```
# Verifier le service
systemctl is-active letrevani-listener

# Tester un notify manuel
psql -d letrevani_validation_final -c \
  "SELECT pg_notify('letrevani_table', '{"event":"test"}');"

```

Module Odoo non trouve apres installation

Activer le mode developpeur Odoo -> Applications -> Mettre a jour la liste

8.2 Verifications rapides

En un coup d'oeil, tout verifier :

```
# 1. Service en route ?
systemctl is-active letrevani-listener

# 2. Sante du listener ?
cat /var/log/letrevani-sync/health.json

# 3. Triggers installes ?
psql -d letrevani_validation_final -c \
"SELECT trigger_name FROM information_schema.triggers
WHERE trigger_name LIKE 'letrevani_%';"

# 4. Module Odoo installe ?
psql -d letrevani_validation_final -c \
"SELECT name, state FROM ir_module_module WHERE name='letrevani_sync';"
```

9. References

Depots Git

- Tadj : https://github.com/hjinaan/letrevani_sync
- Aurelle : https://github.com/aurelle-max/letrevani_sync

Serveur

- Hébergement : letrevani19 (VPN Maysoft 10.8.0.43)
- Odoo : <https://letrevani.com>
- Base : letrevani_validation_final

Composants

- Odoo 19.0 + maj_pos_restaurant_ai (Maysoft)
- PostgreSQL 14+ avec NOTIFY/LISTEN
- Module letrevani_sync v19.0.1.0.0
- Listener Python + service systemd
- 7 triggers PostgreSQL, 3 canaux NOTIFY
- Cle SSH hjinaan : /home/zondy/.ssh/id_ed25519
- Token Aurelle : gh auth (aurelle-max)

Fichiers importants

/maysoft/addons/letrevani_sync/	-> Module Odoo
/opt/letrevani-sync/	-> Listener Python
/var/log/letrevani-sync/	-> Logs et sante
/etc/systemd/system/letrevani-listener.service	-> Service systemd

Document genere par

Imhotep - Zondy - ALM / MAJSOFT

Pour toute question ou incident technique :

support@alm.yt

(c) 2026 - ALM / MAJSOFT - Tous droits reserves